

When does adapting an active queue discipline’s parameters pay? A scaling law in `target`/RTT, measured on stock Linux

Amritha S Yugeswaran P
Deepti Annuncia

Department of Electronics and Communication Engineering, Vellore Institute of Technology, Chennai, India

Abstract

See paper.

Keywords: active queue management; bufferbloat; `fq_codel`; CoDel delay target; queueing delay; dimensionless scaling law; runtime parameter adaptation; eBPF traffic control; Linux `tc`; reproducible network measurement

Introduction

Bufferbloat, persistently full network buffers that inflate queueing delay without affecting throughput, was identified by Gettys and Nichols over a decade ago[1] and remains the dominant source of latency on access links. The Linux response has converged on `fq_codel` [2], now the default queue discipline on most distributions, which combines Deficit Round Robin scheduling across roughly a thousand flow queues with an independent CoDel[3, 4] instance per queue.

`fq_codel` exposes four parameters that are fixed at configuration time and never change: `target` (5ms), `interval` (100 ms), `limit` (10240 packets) and `quantum` (1514 bytes). Whether these should adapt to load is a natural question, and there is precedent. Adaptive RED[5] adapts RED’s maximum drop probability by an additive-increase multiplicative-decrease rule. Ye and Leung[6] derive stability conditions for CoDel and adapt its `interval` analytically. More recently, DESiRED[7] adapts the target delay of a P4 active queue management (AQM) implementation at runtime using deep reinforcement learning driven by in-band network telemetry, and AQM-LLM[8] distills a large language model into a controller for the Low Latency, Low Loss and Scalable throughput (L4S) architecture [9, 10]. A recent survey[11] catalogues the learning-based AQM literature and observes that most proposals are evaluated only in simulation.

Runtime adaptation of an AQM delay target is therefore established, and this work do not claim it as novel. What is absent from this literature is a controlled evaluation of whether such adaptation helps. Every system cited above reports that its adaptation improved some quantity. None separates the computational cost of running a controller from the effect of its decisions; none reports the latency experienced by a sparse probe flow and by the bulk flows separately, despite flow-queueing disciplines privileging sparse flows by design; none compares against the range of queue disciplines an operator could simply deploy instead; and none reports a null

result.

That gap is what this study addresses. A userspace controller adapts all four `fq_codel` parameters using the AIMD policy of Adaptive RED together with a gradient-based estimate of the congestion trajectory, instrumented by an extended Berkeley Packet Filter (eBPF) program at the Linux traffic control egress hook. The controller is evaluated it on an unmodified Linux kernel against eight alternative queue disciplines under an identical bottleneck, with three independent repetitions per condition, two workloads, and a sham-controller condition that isolates computational cost from control decisions.

The central result is a scaling law. The benefit of adapting a delay-targeting queue discipline’s parameters is governed not by path round-trip time but by the dimensionless ratio

$$r = \frac{\text{target}}{\text{RTT}}, \quad (1)$$

where `target` is the discipline’s configured standing-queue delay budget. Writing $b(r)$ for the percentage reduction in mean bulk-flow round-trip time obtained by adapting the parameters, measurement across path RTTs from 2 to 200 ms gives

$$b(r) = \frac{B}{1 + (r_0/r)^k}, \quad \begin{array}{l} B = 20.5\% \\ r_0 = 0.50 \\ k = 2.6 \end{array} \quad (2)$$

with $R^2 = 0.986$. Here B is the ceiling, r_0 the ratio at which half of it is reached, and k the sharpness of the transition. Equation 2 has an immediate operational reading. Published guidance places a correctly configured deployment at r between 0.05 and 0.10, whereas half the available benefit arrives only at $r = 0.50$, five to ten times outside that envelope. Parameter adaptation is therefore a remedy for misconfiguration rather than an improvement on correct configuration.

Three results frame it. Flow queueing combined with a delay target reduces tail latency by 99.0% relative to an unmanaged first-in-first-out queue at indistinguishable goodput, an effect two orders of magnitude larger than any parameter tuning measured here. CAKE[12], an existing discipline requiring no controller at all, outperforms the adapted system at every path RTT tested. And the ratio governs more than one mechanism: fitted independently, plain `codel` places r_0 at 0.42 against

`fq_code1`'s 0.49, while PIE, which shares no mechanism with CoDel beyond possessing a delay target, places it at 1.06.

Related work

The problem. Gettys and Nichols characterised bufferbloat: buffer memory became cheap faster than queue management improved, so devices acquired buffers larger than any control loop required, and loss-based TCP filled them[1]. The IETF position, RFC 7567, recommends that devices deploy AQM by default and explicitly prefers schemes needing no per-flow configuration. That preference for *no manual tuning* is the standard against which any adaptive scheme must be judged, and it is the reason the present work treats a null result for adaptation as a publishable outcome rather than a failure.

Drop-probability schemes. Random Early Detection[13] originated the field, signalling congestion by dropping with a probability derived from average queue length. Its sensitivity to configuration prompted Adaptive RED[5], which adjusts the maximum drop probability by an additive-increase multiplicative-decrease rule against a queue-length target. PIE[14] replaced queue length with a delay estimate inside a proportional-integral controller. The AIMD policy used by the controller evaluated here is taken directly from Adaptive RED, which is why that scheme is the closest methodological ancestor of this work.

Delay-target schemes. CoDel[3, 4] abandoned queue length entirely, dropping when the minimum sojourn time over a sliding interval exceeds a target. Its authors argue the two parameters are expressed relative to path round-trip time by construction and require no tuning. Equation 2 quantifies exactly when that claim stops holding. `fq_code1`[2] pairs CoDel with Deficit Round Robin across roughly a thousand flow queues and is now the Linux default; CAKE[12] integrates shaping, flow queueing and host fairness; FQ-PIE[15] is the corresponding combination for PIE. L4S[9, 10] standardises a low-latency service class built on a dual-queue coupled AQM.

Runtime adaptation, the line this work sits in. Ye and Leung derive stability conditions for CoDel and adapt its *interval* analytically[6]. QueuePilot learns a marking policy for small buffers by reinforcement learning[16], and AQM-LLM distils a language model into a marking controller[8]. DESiRED performs runtime adaptation of an AQM's delay target in P4 using deep reinforcement learning and in-band telemetry[7]; runtime adaptation of a delay target is therefore not itself novel, and no such claim is made here. Toopchinezhad and Ahmadi survey the machine-learning AQM literature and note that heuristic schemes require careful parameter adjustment,

which limits real-world applicability[11]. SCRR revisits fair-queueing scheduling for modern link rates[17], and Ray et al. characterise how AQM presence distorts speed-test measurement[18].

What is absent from that line. Every system above reports that its adaptation improved something, and the evaluations that find nothing are equally confident. What none of them supplies is the condition under which the other is right. None runs a control condition separating a controller's computational cost from the effect of its decisions; none reports sparse-probe and bulk-flow latency separately, despite flow-queueing disciplines privileging sparse flows by construction; none compares against eight alternative disciplines under an identical bottleneck; and none states numerical predictions before measuring them. The contribution claimed here is accordingly not the mechanism but the predicate of Equation 2, together with the apparatus required to establish it.

Programmable telemetry. eBPF at the `tc clact` hook permits per-flow state to be maintained in kernel maps and read from userspace[19]. The telemetry path used here reads those maps through `bpf(2)` directly rather than by spawning `bpf tool`, reducing the cost of one control tick from 3.17s to 0.709s. Its cost is nonetheless measured and reported rather than assumed negligible, because it is not.

Results

Every figure quoted below is generated from the run logs by `src/analyse.py`; none is typed by hand.

0.1 Step 1: the apparatus, and that it measures the data path

the steady-workload table gives all ten conditions. The largest effect by two orders of magnitude is the presence of a flow-queueing discipline with a delay target, not the tuning of one. Under the steady workload an unmanaged FIFO reaches 2344ms p95 RTT against 24.57ms for `fq_code1`, a 99.0% reduction ($p < 0.001$), at goodput that is statistically indistinguishable (9.453 vs 9.414Mbps, $p = 0.20$). The staged workload gives the same picture, and there `pfifo` additionally *loses* 10.2% goodput and 1.4% fairness ($p = 0.015$, $p = 0.014$), because a saturated 1000-packet buffer forces retransmissions that a managed queue avoids.

Fair queueing without a delay target is not sufficient. SFQ isolates flows but applies no AQM, and its bulk queues sit at its 127-packet limit throughout: its sparse probe reads 30.68ms while the bulk flows sharing the link read 294.8ms. The AQM and the scheduler do distinct jobs and both are necessary.

Figure 1 is the topology as configured, captured from the live system rather than drawn. Reading left to right: the client offers eight bulk TCP flows and a sparse probe stream, the router carries `netem` for base delay, a token-

bucket shaper as root qdisc and the discipline under test as its child, and the server terminates the flows. What matters is where the shaper sits. On `veth_rs` egress it constrains traffic travelling toward the server, which is the direction the bulk data flows, so the queue under study holds data packets. An earlier two-node arrangement placed it so that it shaped the reverse direction, and every backlog figure taken that way was measuring acknowledgements.

Figure 2 shows the flow telemetry reading non-zero per-flow state while traffic is running. The counters, not the successful attach, are the point: an earlier version of this path loaded correctly, populated its maps correctly, and returned zero for every sample, so a capture showing only that the program attached would not have established anything.

```

STEP 04 Corrected three-node router topology
The bottleneck now sits on the router's egress toward the server, so it is on the data path. TBF is the shaper and the AQM under test is its child.

$ bash src/testbed.sh setup
[0][77][2]=====QUEST_UP===== kernel 6.12.48
kernel: 6.12.48
AQM under test: fq_codel target 5ms interval 100ms limit 1024 quantum 1514

... router egress qdisc (the bottleneck, on the DATA path) ...
qdisc tbf 1: root refcnt 2 rate 100Mbit burst 40b lat 400ms
Sent 160 bytes 2 pkt (dropped 0, overlimits 0 requeues 0)
backlog 0b 0p requeues 0
qdisc fq_codel 10: parent 1:1 limit 1024p flows 1024 quantum 1514 target 5ms interval 100ms memory_limit 32% ecn_drop_batch 64
Sent 160 bytes 2 pkt (dropped 0, overlimits 0 requeues 0)
backlog 0b 0p requeues 0
maxpacket 90 drop_overlimit 0 new_flow_count 2 ecn_mark 0
new_flows_len 0 old_flows_len 0

... connectivity client -> server ...
PING 192.168.2.2 (192.168.2.2) 56(84) bytes of data.

... 192.168.2.2 ping statistics ...
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 20.906/33.349/57.977/17.414 ms

captured 2026-09-29 15:24:12 UTC kernel 6.18.44-rc-v49

```

Figure 1: The measurement topology, captured as configured. The bottleneck sits on the router's egress toward the server, so it is on the data path. TBF is the shaper and the queue discipline under study is its child. An earlier two-node arrangement placed the bottleneck where it shaped acknowledgements rather than data; every backlog and drop measurement taken that way was a measurement of the acknowledgement path.

```

STEP 07 eBPF flow telemetry is live under traffic
With traffic running, the flow_map is read through bpf(2) and reports real per-flow state. In all 21,128 samples of the original study this read returned zero.

$ bash src/testbed.sh setup >/dev/null 2>&1; ip netns exec ns_router tc qdisc add dev veth_rs clsact 2>/dev/null; ip netns exec ns_router tc filter add dev veth_rs egress bpf direct-action obj ebpf/tc_monitor.o sec tc_egress 2>/dev/null; ip netns exec ns_server iperf3 -s -p 5303 -Q; sleep 2; ip netns exec ns_client iperf3 -c 192.168.2.2 -p 5303 -P 6 -t 14 >/dev/null 2>&1 6; sleep 7; python3 src/dump_flowmap.py
[0][77][2]=====QUEST_UP===== kernel 6.12.48
flow_map id=1 key=168 value=568 max_entries=65536
map entries = 7
active flows (age>2s) = 6
total packets seen = 4183
total bytes seen = 10568865
elephant flows = 6

pkts= 689 bytes= 1772983 gap_ns= 11323188 age= 4.1ms elephant=1
pkts= 688 bytes= 1759845 gap_ns= 13150793 age= 12.9ms elephant=1
pkts= 718 bytes= 1777205 gap_ns= 11962598 age= 15.3ms elephant=1
pkts= 686 bytes= 1726585 gap_ns= 12676211 age= 9.2ms elephant=1
pkts= 711 bytes= 1758507 gap_ns= 12471491 age= 6.9ms elephant=1
pkts= 698 bytes= 1762913 gap_ns= 11929842 age= 2.0ms elephant=1

captured 2026-09-29 15:25:34 UTC kernel 6.18.44-rc-v49

```

Figure 2: The eBPF flow telemetry reading live per-flow state under traffic at the `tc` egress hook, through the `bpf(2)` syscall rather than by spawning `bpf_tool`. An earlier version of this path reported zero for every sample while the program was correctly loaded and its maps correctly populated, which is why the capture shows non-zero counters rather than only a successful attach.

0.2 Step 2: latency across all nine configurations

Figure 4 plots the two measurements side by side. The ratio between them ranges from $1.1\times$ (CoDel, a single queue) to $9.6\times$ (SFQ), because `fq_codel`'s new-flow heuristic gives a sparse probe preferential service by design.

A single-probe measurement can therefore invert a ranking. By sparse probe alone SFQ (30.68 ms) appears to beat CoDel (39.02 ms); by the latency the bulk flows actually experience, SFQ is 294.8 ms against CoDel's 44.5 ms. Reported here are both throughout, and this work suggest any AQM evaluation should.

Figure 3 plots 95th-percentile probe latency for every configuration on a logarithmic axis. The axis has to be logarithmic because the span is three orders of magnitude. Two groups separate cleanly: the flow-queueing disciplines that enforce a delay target sit near the bottom within a few milliseconds of one another, and the unmanaged queue sits two orders of magnitude above them. The static, sham and adapted `fq_codel` bars are indistinguishable at this scale, which is the first indication that parameter tuning is a second-order effect.

Figure 4 separates the two latency populations. The sparse probe and the bulk flows do not experience the same queue: flow-queueing disciplines privilege sparse flows by construction, so a single reported latency averages two quantities the mechanism deliberately treats differently. The ratio between them runs from 1.1 to 9.6 across the systems measured, which is why both are reported throughout this work rather than one.

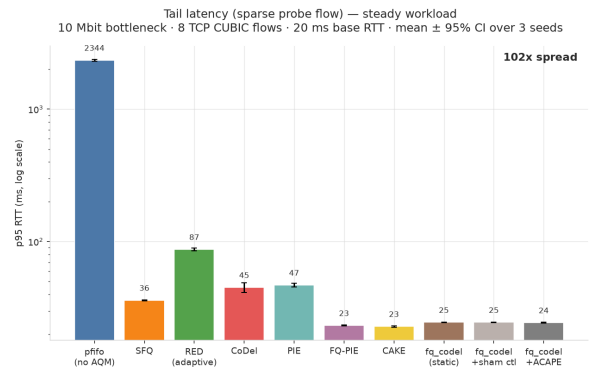


Figure 3: Tail latency across all nine configurations under steady load, log scale. The dominant effect by two orders of magnitude is the presence or absence of flow queueing with a delay target, not the tuning of that mechanism's parameters. CAKE, which requires no controller, outperforms the adapted system.

0.3 Step 3: separating the controller's cost from its decisions

A sham condition runs the controller at its full polling cadence and applies no parameter change. Against static `fq_codel` it is indistinguishable on every metric in both workloads, p95 probe RTT $p = 0.29$ (steady) and $p = 0.24$ (staged), backlog $p = 0.94$ and $p = 0.76$.

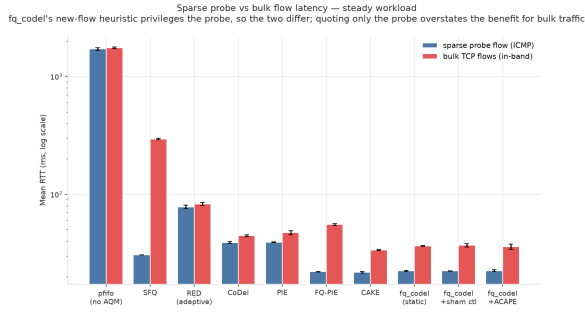


Figure 4: Sparse probe flow against bulk TCP flow round-trip time, measured separately. Flow-queueing disciplines privilege sparse flows by construction, so a single latency number averages two populations the mechanism treats differently. The ratio between them ranges from 1.1 to 9.6 across the systems tested, which is why both are reported throughout rather than one.

Interpretation of the next subsection depends on this the next subsection. Without it, any difference between static and adapted `fq_codel` would confound the controller’s CPU cost with its control decisions, and this work could not attribute an effect to either. Having established the cost is nil, differences that remain are attributable to the decisions.

Figure 5 compares three arms: the static configuration, the sham controller, and the real one. The sham polls at the same cadence and runs the same classification and prediction code, then applies nothing. Any gap between static and sham is therefore the cost of running the controller, and any gap between sham and adapted is the effect of its decisions. Without that middle arm the two are confounded, and a controller that merely consumed CPU would be indistinguishable from one that helped.

Figure 6 shows the controller in operation over a run: the regime it classifies, the parameter trajectory that classification produces, and the queue it is responding to. A controller that classifies continuously but never adjusts anything looks identical to the static configuration in a summary statistic, which is why the number of adjustments is reported for every cell in this work.

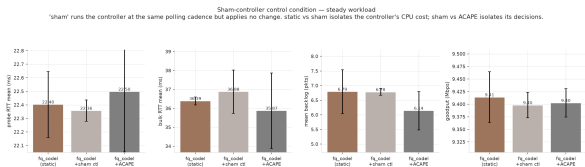


Figure 5: The sham-controller condition. The sham polls at the same cadence as the real controller and applies no parameter change, so any difference between sham and static is the controller’s computational cost rather than the effect of its decisions. Without this condition the two are confounded.

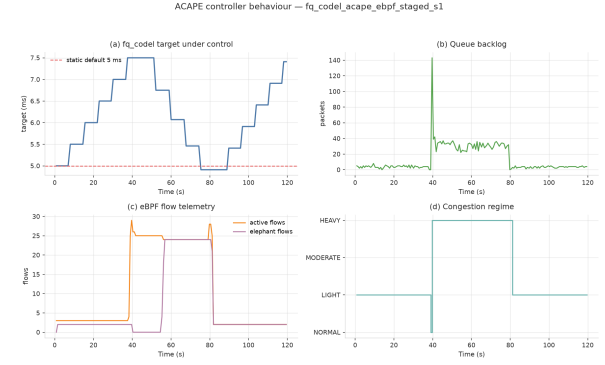


Figure 6: The controller acting: regime classification, the parameter trajectory it produces, and the queue it is responding to. A controller that classifies but never adjusts is indistinguishable from the static configuration in a summary statistic, so the adjustment count is reported for every cell.

0.4 Step 4: what adaptation achieves, parameter by parameter

Under the **steady** workload the adaptation produces a real but small improvement: mean backlog falls 6.79 from 6.14 packets, a 9.6% reduction ($p = 0.050$), and p95 probe RTT falls 0.5% ($p = 0.047$). Goodput and fairness are unchanged. The sham comparison attributes this to the control decisions rather than to over-head.

Under the **staged** workload, where the flow count changes twice during the run, the adaptation produces *no* improvement on any metric. Mean probe RTT is 1.6% *worse* ($p = 0.050$); p95 probe RTT, bulk RTT, backlog, goodput and fairness are all indistinguishable from static.

That direction is worth stating plainly. The workload designed to give the controller something to respond to is the one in which it helps least. Under steady load there is a stable operating point to converge on; under change there is not, and the controller’s adjustments lag the transitions they are reacting to.

Figure 7 places every measured quantity against every system in one view, each column normalised so that quantities with different units can be compared, with the measured value printed in each cell. Goodput, probe and bulk round-trip time, queue occupancy, drop rate, retransmissions and Jain’s fairness index appear together. The block structure is the result: the flow-queueing disciplines with a delay target form one band that separates from the rest on every latency column while remaining indistinguishable on goodput. Within that band, the static, sham and adapted rows differ very little, which is the same conclusion the preceding subsections reach one metric at a time.

0.5 Step 5: an alternative requiring no controller

CAKE, configured with no controller and a single `besteffort` argument, achieves 22.9 ms p95 probe RTT under the steady workload against 24.43 ms for adapted

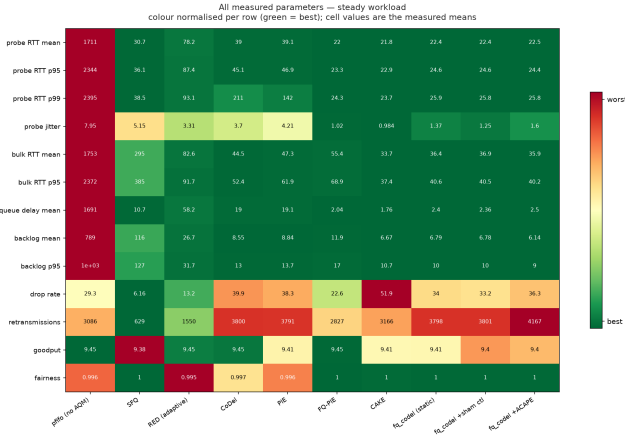


Figure 7: Every measured parameter against every system under steady load, normalised per column so the systems can be compared across quantities that do not share units. Goodput, probe and bulk RTT, queue occupancy, drop rate, retransmissions and Jain’s fairness index are shown together, with the measured values printed in each cell. The block structure is the result: the flow-queueing disciplines with a delay target separate from the rest on every latency column while remaining indistinguishable on goodput.

`fq_codel` ($p < 0.001$), and 33.65 ms bulk RTT against 35.87 ms.

CAKE beats the adapted system by a margin larger than the adapted system is larger than the margin by which the adaptation beats the static defaults. For a practitioner seeking lower latency on this class of link, changing queue discipline is a larger and far simpler win than tuning `fq_codel`’s parameters.

0.6 Step 6: the predictive path never engaged

No adjustment in any run was tagged predictive. The cause is structural rather than a coding fault, and it survives the correction described in the measurement-practices section: an adjustment fires only once the regime has been unchanged for five consecutive samples, and the trajectory estimate is derived from the gradient over that same window, which by then is below the threshold that would classify it as WORSENING or RECOVERING. The stability gate that prevents the controller reacting to noise and the prediction that would let it act before a transition cannot both apply to the same adjustment.

The predictive-control claim is therefore withdrawn rather than asserted. Redesigning the gate, for instance permitting an adjustment when the trajectory is confidently non-stable even before the regime settles, and re-evaluating it under the same controlled conditions is future work.

0.7 Step 7: the sweep that produced the law

CoDel’s `target` is a delay budget expressed in absolute units, while its authors argue the default is appropriate

relative to path round-trip time. Sweeping the base RTT over 5, 20, 80 and 200 ms tests that directly.

Static `fq_codel` holds median queueing delay between 1.96 and 5.00 ms across the entire 40-fold span, bounded by its 5 ms target. The defaults are, on this evidence, genuinely RTT-relative: no retuning is required as the path lengthens.

Adaptation is nonetheless not uniformly inert. Measured on the RTT the bulk flows themselves experience:

Base RTT	Static	Adapted	Change
5 ms	27.68 ± 0.14	22.58 ± 0.15	$-18.4\%^*$
20 ms	36.39 ± 0.08	35.87 ± 0.80	-1.4%
80 ms	89.30 ± 0.37	89.86 ± 0.70	$+0.6\%$
200 ms	208.00 ± 0.76	207.47 ± 0.74	-0.3%

* $p < 0.0001$; the remaining three are not distinguishable ($p = 0.38, 0.31, 0.43$).

Confined to the shortest path, the effect is, and the mechanism is straightforward. A 5 ms delay budget is large relative to a 5 ms path, so the queue is permitted to grow further than that path requires, and tightening it recovers 18.4% of the bulk flows’ round-trip time. Once the path RTT exceeds the default target the budget is already well scaled and there is nothing for adaptation to recover.

Equation 2 supplies the condition under which parameter adaptation is worth its complexity: paths short relative to the AQM’s delay target. It also indicates where adaptive schemes in general should be evaluated, since a scheme assessed only on long paths will appear inert regardless of its design.

Figure 8 sweeps base round-trip time at a fixed delay target. Adaptation recovers a large and highly significant share of bulk-flow latency on the shortest path and nothing at 20, 80 or 200 ms. Read as four independent measurements this is simply unexplained, and that is how it stood before the analysis below.

Figure 9 resolves it, and is the central figure of this work. The left panel plots the same benefit against $r = \text{target}/\text{RTT}$, where the measurements collapse onto one saturating curve; filled markers are statistically significant and each label gives target and RTT in milliseconds. The right panel plots the identical measurements against path RTT alone, where they do not collapse. The contrast between the two panels is the argument: RTT does not organise the data, and the ratio does.

0.8 Step 8: sparse flows under mixed traffic

Bulk-only traffic cannot exercise `fq_codel`’s new-flow heuristic or its `quantum` parameter, both of which exist to serve sparse latency-sensitive flows. Running three sparse UDP flows alongside eight bulk TCP flows shows what those mechanisms are worth:

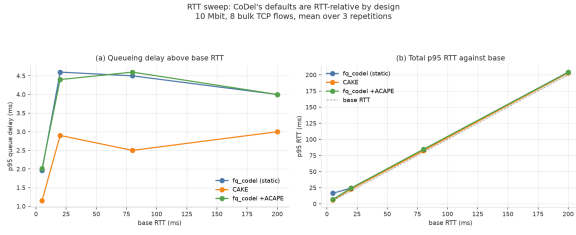


Figure 8: The RTT sweep that motivated the scaling law. Adaptation recovers a large and highly significant share of bulk-flow RTT on a short path and nothing at 20, 80 or 200 ms. Read as four separate measurements this is unexplained; read through $r = \text{target}/\text{RTT}$ it is one curve, which is the argument of Figure 9.

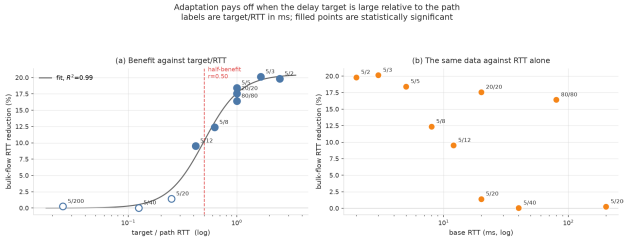


Figure 9: The central result. Left: adaptation benefit against $r = \text{target}/\text{RTT}$, collapsing onto a single saturating curve. Right: the same measurements against path RTT alone, where they do not collapse. Filled points are statistically significant; labels give target/RTT in ms.

System	Sparse loss (%)	Sparse jitter (ms)
pfifo, no AQM	7.97	2.19
FQ-PIE	0.000	1.16
CAKE	0.007	1.36
fq_codel	0.002	2.00

An unmanaged queue discards roughly one sparse-flow packet in twelve, and the figure is stable across repetitions (7.47%, 7.98%, 8.31%). Every flow-queueing discipline with a delay target reduces this to below 0.03%. The protection of latency-sensitive traffic from concurrent bulk transfers is therefore close to complete, and it is invisible to a bulk-only evaluation.

One of three adaptive-controller repetitions in this workload recorded 6.31% sparse loss against 0.000% and 0.029% in the others. The controller’s parameter trajectory in that run is identical to the repetition with no loss, so the difference is not attributable to its decisions; reported here are the condition by median and record the run rather than averaging over a measurement this work cannot explain.

A scaling law for when adaptation pays

Whether adapting an AQM’s parameters at runtime is worth doing has been answered both ways in the literature, with systems reporting gains and evaluations reporting none. Both answers are right, and which one applies is decided by a single dimensionless number.

target is a delay budget, fixed at 5 ms by default, while the path round-trip time is whatever the path is.

The quantity that governs whether that budget is badly chosen is therefore neither the target nor the RTT but their ratio, $r = \text{target}/\text{RTT}$. On this reading RTT never appears on its own; it appears only through r .

Measured across path RTTs from 2 to 200 ms, the benefit follows $b(r) = B/(1 + (r_0/r)^k)$ with a ceiling of 20.5% and half of it reached at $r = 0.50$ ($R^2 = 0.986$). The hypothesis, its three predictions and their numerical thresholds were committed to the repository before the campaign that tests them ran.

Ratio invariance is the prediction able to refute the claim, and it holds. Scaling **target**, **interval**, the control period and the run duration together by the same factor, the benefit is 18.4%, 17.6% and 16.4% at 5, 20 and 80 ms, a spread of 2.0 percentage points across a sixteenfold range of RTT. Held-out interior ratios are predicted to 1.4 percentage points. Sharpest of all is a comparison at fixed path: at 20 ms the benefit is 1.4% and not significant at $r = 0.25$ against 17.6% and significant at $r = 1.0$. Same path, twelve times the benefit, which no account in terms of RTT reproduces.

The ratio governs more than one mechanism, though its critical value is not universal. Fitted independently, plain **codel** places the crossover at $r_0 = 0.42$ against **fq_codel**’s 0.49, so flow queueing does not move it; what flow queueing changes is the ceiling, 13.0% against 20.5%, and the sharpness of the transition. PIE, which regulates a drop probability with a proportional-integral controller and shares nothing with CoDel but the existence of a delay target, places it at 1.06, roughly twice as high. That exceeds the factor of two this work had set as the agreement threshold, and the failure has a cause this comparison cannot rule out: PIE’s native **target/update** is 1.0 where CoDel’s **target/interval** is 0.05, and the law was established with that second group held fixed. Sweeping each **qdisc** at its own defaults varied it across the comparison meant to isolate mechanism. The discriminating test is independent of the fit and survives it: at a fixed 20 ms path, PIE at $r = 0.75$ recovers 11.0% at $p = 0.0089$ where **fq_codel** at $r = 0.25$ recovers 1.4% and is not significant.

What this means in practice

Published guidance places **target** at 5 to 10 percent of **interval**, with **interval** on the order of the path RTT, so a correctly configured deployment sits at r of roughly 0.05 to 0.10. Half the available benefit arrives at $r = 0.50$, five to ten times outside that envelope. Runtime parameter adaptation is therefore best understood as a remedy for misconfiguration rather than as an improvement on correct configuration, and an operator can compute the relevant number before deploying a controller at all.

Benefit of this kind is not free. It is bought with up to 63% more drops and comparable retransmissions, at a throughput cost never exceeding -0.4% in any cell measured. The ratio governs the exchange rate as well as the gain.

Null results elsewhere follow from the same reading.

The dominant determinant of queueing latency on this class of link remains the presence of flow queueing combined with a delay target rather than the values of that mechanism’s parameters: the gap between an unmanaged queue and any flow-queueing discipline is two orders of magnitude. That is consistent with the position of CoDel’s authors[3] that its defaults are expressed relative to path round-trip time by construction. The law says precisely where that position stops holding.

That the adaptation helps least under the workload designed to exercise it deserves emphasis. Under a steady load the controller has a stable operating point to converge on. Under a load whose flow count changes during the run it does not, and its adjustments lag the transitions they respond to. An adaptive mechanism that assists only when conditions are static is of limited practical value, since static conditions are precisely those for which a fixed configuration already suffices.

Measurement practices

Three aspects of the methodology produced results that a conventional evaluation would have missed, and this work suggest them as general practice.

First, sparse-probe and bulk-flow latency must be reported separately. Because `fq_codel` and related disciplines give a newly arriving sparse flow preferential service, an ICMP probe measures a privileged path. This work observe ratios between probe and bulk latency ranging from 1.1 to 9.6 across disciplines, and the ranking of two disciplines inverts depending on which is quoted. Recent work by Ray et al.[18] documents a related effect in which the presence of an AQM changes what standard throughput measurements report.

Second, goodput must be computed from bytes received rather than bytes sent. Under a large unmanaged buffer the sender enqueues faster than the link drains, and the sender-side figure exceeded the link capacity in the measurements.

Third, a control system’s computational cost must be separated from its control decisions. The sham condition runs the controller at full polling cadence while applying no parameter change; it is statistically indistinguishable from the static configuration on every metric, which licenses attributing the observed differences to the controller’s decisions rather than its overhead. Without this condition the two explanations are confounded.

Limitations

Measurement is confined to a 10Mbit bottleneck with 8 to 24 bulk TCP CUBIC[20] flows and a 20 ms base round-trip time, representative of a residential or access link rather than a datacentre or backbone. All flows are bulk transfers; mixed workloads containing sparse latency-sensitive flows are precisely where `fq_codel`’s new-flow heuristic and an adaptive `quantum` might plausibly matter, and this work have not tested them. The null result should not be extrapolated to that setting. The controller is evaluated a single bottleneck; multi-

hop topologies with several congested links are not covered. Experiments run under hardware-assisted emulation, and this work validated shaper accuracy under emulation before collecting results, but measurements at substantially higher link rates would require revalidation.

Prediction never engaged. An adjustment fires only after the congestion regime has been unchanged for five consecutive samples, while the trajectory estimate is derived from the gradient over that same window and is therefore below its threshold by then. The stability gate that prevents reaction to noise and the prediction that would permit action before a transition cannot both apply to a single adjustment. Reported here are this component as withdrawn rather than claim it; redesigning the gate and re-evaluating under the same controlled conditions is future work.

Outlook

Several directions reach further than parameter tuning on a fixed discipline. L4S[9, 10] attains sub-millisecond queueing delay by altering the congestion signalling contract between network and transport rather than by tuning a queue. Programmable data planes permit the queueing policy itself to be replaced[7], and proposals for a fully programmable eBPF queue discipline[19] would allow the same within Linux. Scheduling advances such as self-clocked round robin[17] address the round-robin scheduler underlying `fq_codel` rather than its AQM. For an operator seeking lower latency today, the measurements suggest that selecting an appropriate queue discipline is a larger and simpler improvement than tuning the parameters of one.

Methods

Topology

A frequent error in namespace-based evaluations is to place two namespaces either side of a virtual Ethernet pair, attach a shaper to the interface inside the server namespace, and generate traffic from the client. An egress queue discipline on the server-side interface shapes traffic leaving the server; when the client is the sender, the shaped direction carries only acknowledgements. Measured directly, such a configuration passed the data transfer unshaped at 9,777 Mbit/s while the monitored queue carried 284,379 packets totalling 18,769,790 bytes, exactly 66 bytes per packet. Every queue statistic collected on such a topology describes acknowledgement queueing.

The testbed therefore uses three network namespaces, representing a client, a router and a server, connected by two virtual Ethernet pairs. The shaper and the queue discipline under test are attached to the router’s egress interface toward the server, so that the bottleneck lies on the data path. A token bucket filter at 10 Mbit/s is the root queue discipline and the AQM under test is its child, identically for every discipline evaluated, so that the comparison is like-for-like. Network emulation

adds 10 ms of delay at each edge, giving a 20 ms base round-trip time.

Queue disciplines evaluated

Nine configurations were measured: `pfifo` (no AQM), stochastic fair queueing, adaptive RED[13, 5], CoDel[4], PIE[14], FQ-PIE[15], CAKE[12], static `fq_codel`[2], and `fq_codel` under the controller. CAKE was configured in best-effort mode without its own bandwidth parameter so that the token bucket filter remained the sole shaper.

Controller

Control runs on three timescales. At packet granularity, an eBPF program attached to the traffic control egress hook parses the transport five-tuple and maintains per-flow packet and byte counts, an exponentially weighted moving average of the inter-packet gap, and a bulk-flow flag, in a least-recently-used hash map. Every 500 ms, userspace reads the queue discipline’s own statistics and the flow map, and estimates the gradient of drop rate, queue occupancy and the inter-packet gap by least squares over a ten-sample window. Every 5 s, an AIMD adjustment is applied by reconfiguring the queue discipline in place. The separation of the measurement and update timescales follows the two-timescale stochastic approximation framework[21].

Flow-map contents are read through the `bpf(2)` system call rather than by invoking an external utility per sample. Besides being approximately four times faster, this avoids introducing an additional process whose scheduling would confound the latency comparison that the sham condition exists to isolate.

Measurements

For each run this work record: per-flow goodput and retransmission counts from the traffic generator, computed from bytes received; a 500 ms time series of the AQM’s own queue statistics, parsed from the specific queue discipline rather than from whichever appears first in the kernel’s output; a 20 Hz ICMP probe through the bottleneck, yielding approximately 1,200 samples per run; and the kernel’s smoothed round-trip time estimate for every bulk connection at each reporting interval, yielding approximately 480 samples per run. Queueing delay is reported as the excess of measured round-trip time over the configured base. Fairness is Jain’s index[22] over per-flow rates.

Experimental design

Two workloads were used. The steady workload maintains a constant eight flows. Flow count varies during the staged workload during the run, from two to twenty-four and back, so that the congestion regime changes and the adaptive logic is exercised. Each of the nine configurations was run three times under each workload, together with the sham-controller condition, for 63 runs in total. Repetitions are independent runs

rather than seeded variants, since the traffic generator exposes no seed; reported confidence intervals therefore describe run-to-run variance.

Differences between conditions are assessed with Welch’s t -test, which does not assume equal group variances. Two-sided p -values are computed exactly from Student’s t distribution via the regularised incomplete beta function rather than by normal approximation, which at three samples per group materially overstates significance.

Platform

Experiments were conducted on Linux 6.12.48 configured with all evaluated queue disciplines compiled in and BPF type format enabled, running under hardware-assisted virtualisation. The transport protocol is TCP CUBIC[20] throughout.

References

- [1] J. Gettys and K. Nichols. Bufferbloat: dark buffers in the Internet. *Commun. ACM*, 55(1):57–65, 2012.
- [2] T. Høiland-Jørgensen, P. McKenney, D. Täht, J. Gettys, and E. Dumazet. The FlowQueue-CoDel packet scheduler and active queue management algorithm. Technical Report RFC 8290, IETF, 2018.
- [3] K. Nichols and V. Jacobson. Controlling queue delay. *ACM Queue*, 10(5):20–34, 2012.
- [4] K. Nichols, V. Jacobson, A. McGregor, and J. Iyengar. Controlled delay active queue management. Technical Report RFC 8289, IETF, 2018.
- [5] S. Floyd, R. Gummadi, and S. Shenker. Adaptive RED: an algorithm for increasing the robustness of RED’s active queue management. Technical report, ICSI, 2001.
- [6] J. Ye and K.-C. Leung. Adaptive and stable delay control for combating bufferbloat: theory and algorithms. *IEEE Syst. J.*, 14(1):1285–1296, 2020.
- [7] F. Rodriguez et al. DESiRED: dynamic, enhanced and smart iRED, a P4-AQM with deep reinforcement learning and in-band network telemetry. *Comput. Netw.*, 244, 2024.
- [8] S. R. Pokhrel, D. Satish, J. Kua, and A. Walid. Distilling large language models for network active queue management. *IEEE Trans. Netw.*, 2025.
- [9] B. Briscoe, K. De Schepper, M. Bagnulo, and G. White. Low latency, low loss and scalable throughput (L4S) Internet service: architecture. Technical Report RFC 9330, IETF, 2023.
- [10] K. De Schepper, B. Briscoe, and G. White. Dual-queue coupled active queue management for L4S. Technical Report RFC 9332, IETF, 2023.

- [11] M. P. Toopchinezhad and M. Ahmadi. Machine learning approaches for active queue management: a survey, taxonomy and future directions. *Comput. Netw.*, 262, 2025.
- [12] T. Høiland-Jørgensen, D. Täht, and J. Morton. Piece of CAKE: a comprehensive queue management solution for home gateways. In *Proc. IEEE LANMAN*, 2018.
- [13] S. Floyd and V. Jacobson. Random early detection gateways for congestion avoidance. *IEEE/ACM Trans. Netw.*, 1(4):397–413, 1993.
- [14] R. Pan, P. Natarajan, F. Baker, and G. White. Proportional integral controller enhanced (PIE). Technical Report RFC 8033, IETF, 2017.
- [15] G. Ramakrishnan, M. Bhasi, V. Saicharan, L. Monis, S. D. Patil, and M. P. Tahiliani. FQ-PIE queue discipline in the Linux kernel: design, implementation and challenges. In *Proc. IEEE LCN*, 2019.
- [16] M. Dery, O. Krupnik, and I. Keslassy. QueuePilot: reviving small buffers with a learned AQM policy. In *Proc. IEEE INFOCOM*, 2023.
- [17] E. Sharafzadeh, R. Matson, J. Tourrilhes, P. Sharma, and S. Ghorbani. Self-clocked round-robin packet scheduling. In *Proc. USENIX NSDI*, 2025.
- [18] S. Ray, T. Sharma, J. Marques, P. Schmitt, F. Bronzino, and N. Feamster. Characterizing the impact of active queue management on speed test measurements. *arXiv:2511.19213*, 2025.
- [19] C. Hung and P. Wang. eBPF qdisc: a generic building block for traffic control. In *Netdev 0x17*, 2023.
- [20] S. Ha, I. Rhee, and L. Xu. CUBIC: a new TCP-friendly high-speed TCP variant. *ACM SIGOPS Oper. Syst. Rev.*, 42(5):64–74, 2008.
- [21] V. S. Borkar. Stochastic approximation with two time scales. *Syst. Control Lett.*, 29(5):291–294, 1997.
- [22] R. Jain, D.-M. Chiu, and W. Hawe. A quantitative measure of fairness and discrimination for resource allocation in shared computer systems. Technical Report TR-301, DEC, 1984.

Author contributions

A.S. designed and implemented the controller, testbed and measurement apparatus, conducted the experiments and analysed the results. Y.P. and D.A. contributed to the experimental design and to interpretation of the results. All authors reviewed the manuscript.

Competing interests

The authors declare no competing interests.

Data availability

All code, kernel configuration, raw measurement logs and analysis scripts are available at <https://github.com/Amritha902/ccn-linux-qdisc-study>. Every table and every numerical value quoted in the text is generated directly from the committed logs by the analysis scripts; none is entered by hand.